



Trabalho de INF1636

06/05/2026

Prof. Ivan Mathias Filho

Instruções para a 2ª iteração

Na 2ª iteração será iniciada a implementação de uma interface gráfica por meio da qual se possa jogar partidas de Clue.

Interface Gráfica

A interface gráfica do jogo tem de usar componentes **Java Swing** e **Java2D**. O tabuleiro, os piões e as cartas têm de ser exibidos, **OBRIGATORIAMENTE**, por meio da **API JAVA 2D**. Serão disponibilizados arquivos com imagens desses elementos de jogo na página da disciplina no EAD.

É importante ressaltar que **não serão aceitas** soluções em que as imagens são exibidas por meio de componentes **Java Swing**, tais como **JPanel**, **JButton**, **JLabel** e etc. As imagens relativas aos elementos de jogo TÊM DE SER EXIBIDAS por meio do método **drawImage()**, definido na classe **Graphics2D**.

As janelas criadas para o jogo devem ter dimensões máximas de 1400 (largura) por 1050 (altura) pixels.

Janelas Iniciais

A primeira janela (Figura 1) permitirá que os jogadores escolham entre começar uma nova partida ou continuar uma partida cujo estado foi salvo em um arquivo.



Figura 1 – Exemplo de 1ª janela

Feita a escolha de uma nova partida, a janela inicial deverá ser fechada e uma nova janela deverá ser exibida. Essa janela (Figura 2) permitirá que sejam escolhidos os personagens cujos papéis serão assumidos pelos jogadores.

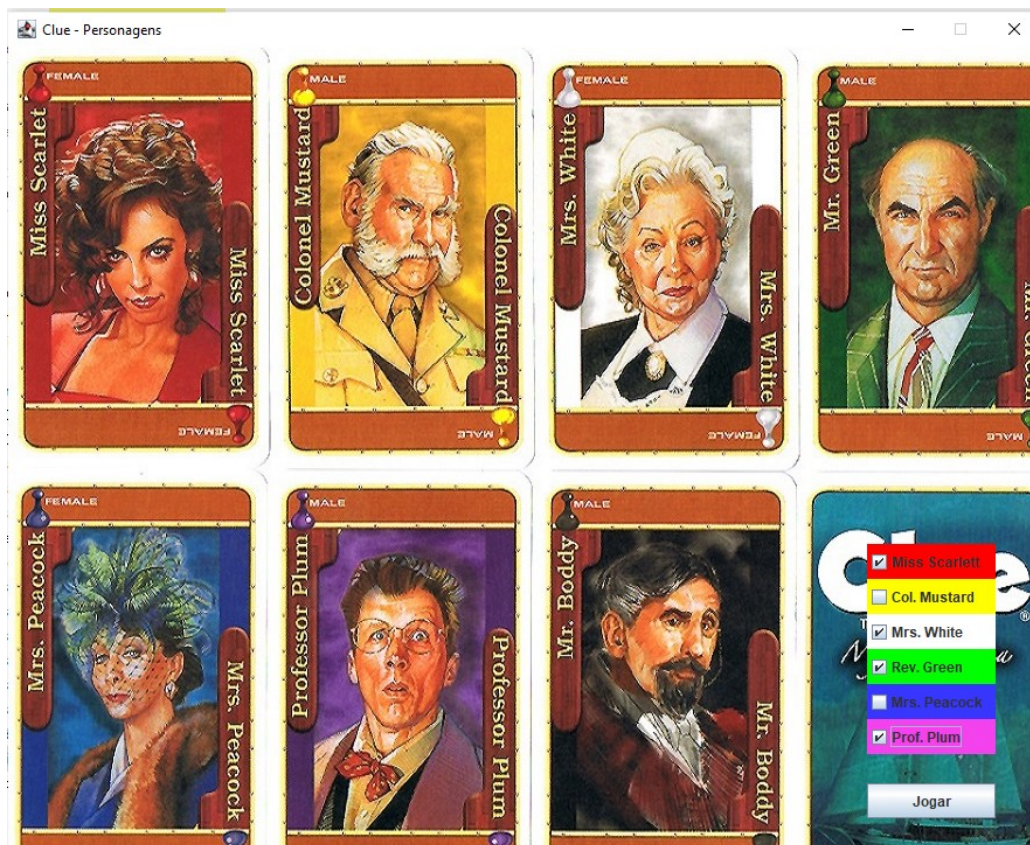


Figura 2 – Exemplo de janela para a escolha dos personagens

Feita as escolhas, a segunda janela deverá ser fechada e a janela relativa ao tabuleiro deverá ser exibida.

Os grupos poderão usar os componentes **Java Swing** que acharem mais adequados para implementarem as funcionalidades das janelas iniciais.

O Tabuleiro

A imagem do tabuleiro (Figura 3) do jogo **NÃO** pode ser exibida por meio de componentes **Java Swing**. **APENAS** o método **drawImage()**, pertencente à classe **Graphics2D**, pode ser usado.

Componentes **Java Swing** poderão ser usados nas seguintes situações:

- Salvamento do jogo corrente (**JPopupMenu**, **JMenu** ou **JButton**);

- Simulação do lançamento dos dados (**JButton**);
- Feitura de palpite ou de acusação (**JRadioButton**, **JCheckBox**, **JList** ou **JComboBox**).



Figura 3 – O tabuleiro

As Jogadas

Como regra geral, todas as operações que não precisarem de intervenção do jogador terão de ser feitas automaticamente pelo programa.

A interface gráfica deve indicar, de maneira clara, a cor do jogador da vez. Isso pode ser feito por meio de uma mensagem exibida no próprio tabuleiro ou por meio de cores. A Figura 4 ilustra como é possível indicar o jogador da vez, pintando a área sobre a qual os dados são exibidos com a cor desse jogador (por exemplo, vermelho).

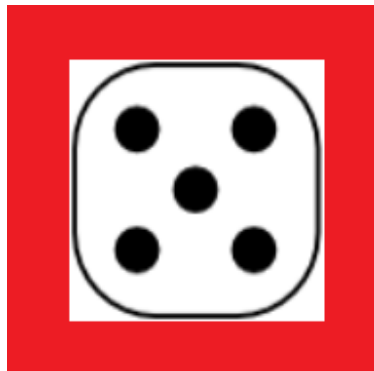


Figura 4 – Indicação do jogador da vez

Evite usar caixas de diálogo (**JDilog**, **JOptionPane** e etc.) para exibir simples mensagens, pois elas exigem a intervenção do jogador, o que atrapalha o andamento das jogadas.

O resultado da simulação do lançamento dos dados deve ser exibido por meio de figuras que representem os números obtidos no lançamento. Todas as seis figuras foram publicadas na página da disciplina no EAD.

Observação: para testar o comportamento do jogo em situações específicas é desejável que os valores dos dados possam ser definidos pelo testador, em vez de serem definidos por meio de uma função de randomização. Sendo assim, é desejável, já na 2ª iteração, que seja implementada uma forma de definição, pelo testador, dos valores dos dados. **Sugestão:** use dois **JComboBox** que contenham os inteiros de 1 a 6.

Deslocamento dos Piões

Após lançar os dados o jogador deverá executar um clique sobre o tabuleiro para indicar para qual casa ou cômodo deseja se deslocar. Caberá ao programa permitir ou não o deslocamento indicado pelo jogador.

Exibição das Cartas dos Jogadores

As cartas que estão com um jogador deverão ser exibidas em uma janela própria (Figura 5). Essa janela deverá ser aberta e fechada por meio de um componente **Java Swing**.

As imagens das cartas farão parte do componente View da arquitetura MVC. O componente Model deverá representar, apenas, o significado de cada carta. Além disso, é desejável que cada carta presente no Model possua um identificador que permita à View recuperar a imagem correspondente em tempo constante (por exemplo, um índice de um array ou uma chave de um Hash Map).

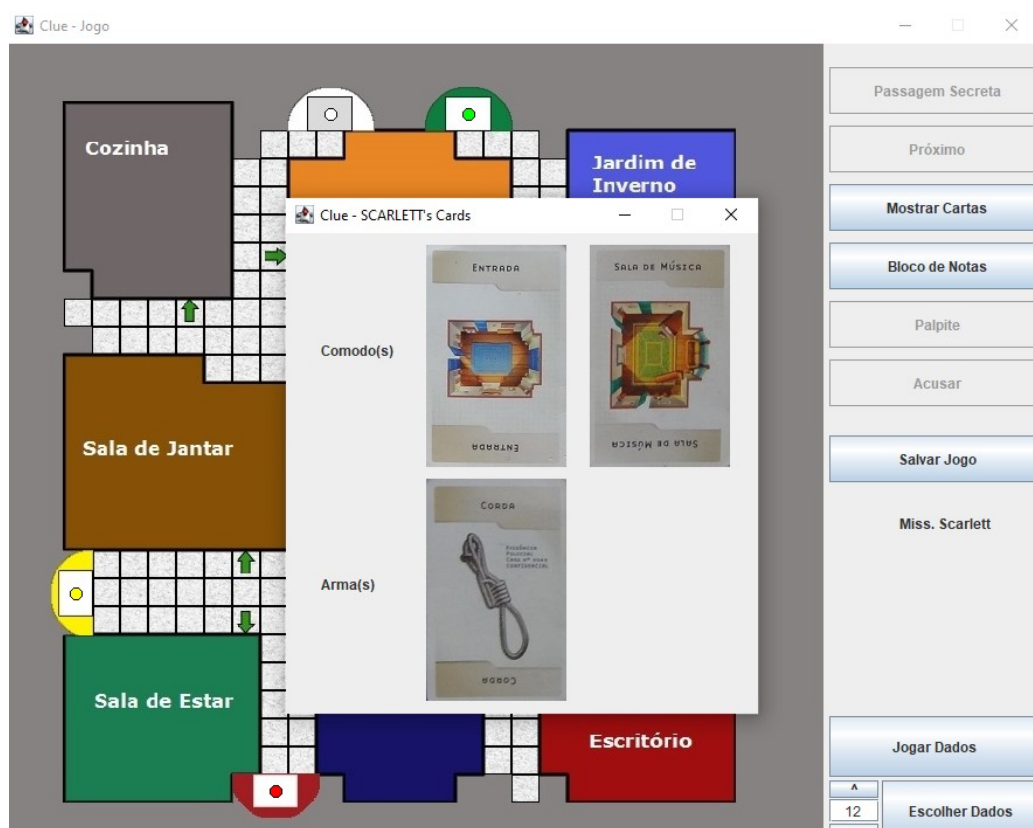


Figura 5 – Exibição das cartas

Exibição e Atualização do Bloco de Notas

O bloco de notas de um jogador deverá ser exibido em uma janela própria (Figura 6). Essa janela deverá ser aberta e fechada por meio de um componente **Java Swing**.

Nesse bloco de notas deverão ser marcadas, automaticamente, as cartas que estão de posse desse jogador. Além disso, ele deverá permitir que o jogador atualize as marcações à medida que seus adversários lhe mostrem cartas que estão de posse deles.

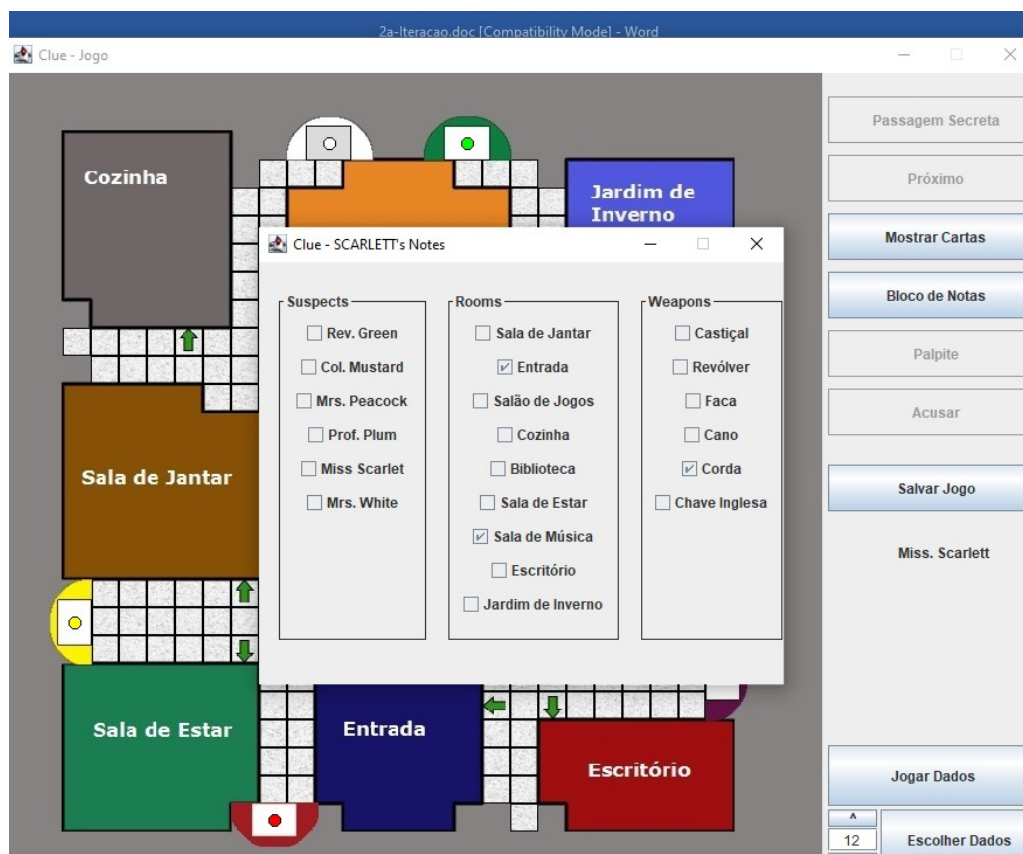


Figura 6 – Exibição do bloco de notas

Objetivo da 2ª iteração

Ao término desta iteração será disponibilizada uma versão executável do jogo na qual os piões dos personagens poderão se deslocar de acordo com os valores obtidos na simulação do lançamento dos dados, além de poderem mudar de cômodo por meio das passagens secretas. Os deslocamentos terão de atender a todas as regras do jogo.

Não será necessário exibir as janelas iniciais, a janela com as cartas e a janela relativa ao bloco de notas.

Os deslocamentos dos piões devem ser implementados da seguinte maneira: após o lançamento dos dados o jogador deverá realizar um clique sobre uma casa ou sobre um cômodo. Caso o deslocamento seja possível, o pião da vez deverá ser deslocado para o local apontado pelo jogador. Caso contrário, nada deverá ser feito. Nenhuma mensagem deverá ser exibida e não haverá mudança do jogador da vez.

Implementações diferentes da descrita acima, principalmente as que usarem botões ou as setas de direção para auxiliar os deslocamentos, serão aceitas, mas o grupo que empregar tais métodos será punido com a perda de 1,0 ponto na nota final.

Vídeo

É importante que o vídeo da 2ª iteração demonstre a execução de jogas, tais como descritas nos objetivos da interação. Ele deve, também, relatar os problemas encontrados e os motivos pelos quais uma ou outra funcionalidade não foi implementada. Por último, esse vídeo deve apontar as funcionalidades que serão implementadas na próxima iteração.

Testes Unitários

Não será necessário escrever novos testes unitários.

Expressões Lambda

Uma expressão lambda é um pequeno bloco de código que recebe parâmetros e retorna um valor. As expressões lambda são semelhantes a métodos, mas não precisam de um nome e podem ser escritas diretamente dentro do corpo de um método.

Exemplo:

```
numbers.forEach((n) -> { System.out.println(n); });
```

Observação: É PROIBIDO O USO DE EXPRESSÕES LAMBDA NO TRATAMENTO DE EVENTOS DE COMPONENTES GRÁFICOS. Todo tratamento de eventos deve ser feito como está apresentado no capítulo 12. **O uso de expressões lambda acarretará na perda de 2,0 na nota final do trabalho.**